CAHIER DES CHARGES

Application de synchronisation de playlists musicales

- BeatSync App -

1. Présentation du projet

Nom du projet : YouTube to Spotify - BeatSync App
Objectif principal :
Permettre à un utilisateur de **synchroniser automatiquement ses playlists YouTube vers Spotify**, avec la possibilité d'étendre ultérieurement le service à d'autres plateformes (Deezer, etc.).

2. Contexte et motivation

Les utilisateurs de plateformes musicales ont souvent des playlists éparpillées sur différents services. Recréer manuellement une playlist d'une plateforme à une autre est fastidieux.
Ce projet vise à simplifier cette opération en automatisant la migration des playlists via les **API officielles** (YouTube Data API, Spotify Web API…).

3. Objectifs fonctionnels

Fonctionnalités principales

1. **Connexion sécurisée via OAuth 2.0**
   - Authentification de l'utilisateur sur Spotify.
   - Gestion sécurisée des tokens OAuth côté backend.
2. **Lecture des playlists YouTube**
   - Récupération du nom, de la description et de la liste des morceaux d'une playlist YouTube publique accessible via URL
3. **Création d'une playlist Spotify correspondante**
   - Création automatique sur le compte utilisateur.
   - Recherche des morceaux correspondants sur Spotify (par titre / artiste).
   - Ajout des morceaux trouvés.
4. **Interface utilisateur simple et claire (React)**
   - Page d'accueil avec champ d'URL YouTube.
   - Page d'état de la synchronisation (progression, erreurs éventuelles).
5. **Notifications de succès / échec**
   - Message de confirmation une fois la synchronisation terminée.

- Gestion des cas d’échec (morceaux introuvables, playlists privées, etc.).

3.1 Périmètre de la version initiale (MVP)

La version initiale de BeatSync correspond à un MVP (Minimum Viable Product) dont l’objectif est de proposer une synchronisation fiable et sécurisée entre YouTube et Spotify, sur un périmètre maîtrisé.

Fonctionnalités incluses dans la version initiale :

- Authentification sécurisée via OAuth 2.0 Spotify
- Stockage sécurisé des tokens côté backend (MongoDB)
- Synchronisation d’une playlist YouTube publique via son URL
- Extraction des titres (nom + artiste lorsque disponible)
- Recherche des morceaux sur Spotify avec logique de correspondance (titre / artiste)
- Création automatique d’une playlist sur le compte Spotify de l’utilisateur
- Ajout des morceaux trouvés
- Affichage d’un récapitulatif (morceaux trouvés / non trouvés)
- Enregistrement d’un historique des synchronisations (PostgreSQL)

Ce périmètre volontairement restreint permet d’assurer :

- la stabilité de l’application,
- la conformité avec les politiques des APIs utilisées,
- une architecture claire et évolutive.

3.2 Hors périmètre (versions ultérieures)

Les fonctionnalités suivantes ne sont pas incluses dans la version initiale, mais constituent des pistes d’évolution :

- Synchronisation des playlists YouTube privées (nécessiterait l’implémentation de l’authentification OAuth Google et la gestion de scopes supplémentaires).
- Synchronisation bidirectionnelle (Spotify → YouTube).
- Sélection et synchronisation de plusieurs playlists simultanément.
- Support d’autres plateformes musicales (Deezer, Tidal, etc.).
- Amélioration avancée de l’algorithme de correspondance des morceaux (scoring intelligent, gestion fine des remasters, live, versions alternatives).

4. Objectifs techniques

Stack technique

- **Frontend :** React.js
- **Backend :** Python (Flask)
- **Tests automatisés :** Pytest + Workflow GitHub Actions

- **Tests manuels :** Postman (vérification des endpoints REST (authentification, création de playlists, envoi de requêtes API)) et Interface utilisateur (tests manuels de parcours complets sur le frontend React)
- **APIs utilisées :**
  - YouTube Data API v3
  - Spotify Web API
- **Hébergement prévu :**
  - Frontend → Vercel
  - Backend → Render ou Railway

Architecture

- **Frontend (React)** : interface utilisateur, requêtes REST vers le backend.
- **Backend (Flask)** :
  - Gestion des appels API (YouTube / Spotify).
  - Conversion et synchronisation des playlists.
  - Gestion de l'authentification et des tokens.
- **Tests automatisés :** via GitHub Actions (CI/CD).

5. Contraintes

- Respect des politiques d'utilisation des API officielles.
- Gestion sécurisée des tokens OAuth côté backend.
- Respect des contraintes OAuth (URI de redirection, HTTPS en production).
- Compatibilité navigateur : Chrome, Firefox, Edge.
- Code clair et documenté pour faciliter l'évolution future.

6. Design et ergonomie

- Palette de couleurs apaisante et moderne (vert / rose pastel / blanc).
- Logo circulaire symbolisant la musique et la synchronisation (flèches + disque).
- Animation de cœur battant ("Let's Beat in Sync") sur la page d'accueil.

7. Planning prévisionnel

Étape	Description	Statut
Analyse du besoin	Définition du périmètre et des APIs	Fait
Conception technique	Architecture frontend/backend	Fait
Développement backend	Développement des endpoints REST et tests unitaires	Fait
Développement frontend	React + intégration des endpoints backend	Fait
Workflow CI/CD	Tests automatisés sur GitHub Actions	En cours de stabilisation
Tests utilisateurs	Scénarios réels de synchronisation	À venir
Déploiement final	Mise en ligne sur Docker/Render	À venir

8. Évolutivité et maintenance

Le code est conçu pour être extensible :

- Tests unitaires pour assurer la stabilité du code.
- Utilisation de GitHub Actions pour détecter rapidement les régressions.

9. Auteure du projet

Nom : Ruken Dogan
Rôle : En formation Développeur Full-Stack
Statut : Projet individuel d’apprentissage

10. Évolutions techniques et décisions d’architecture

Au cours du développement, plusieurs choix techniques ont évolué afin d’améliorer la sécurité, la robustesse et l’évolutivité de l’application. Ces ajustements traduisent une montée en maturité du projet et une meilleure structuration de son architecture.

1. Évolution UX

- Passage d’un fond vert initial à une palette bleutée.
- Amélioration du confort visuel et cohérence graphique globale.

2. Gestion des tokens

- Version initiale : stockage temporaire simple.
- Version actuelle : stockage sécurisé côté backend via MongoDB.
- Implémentation du refresh token pour éviter les réauthentifications répétées.
- Tokens jamais exposés côté frontend.

3. Base de données

- Introduction d’une architecture hybride :
  - MongoDB : gestion des tokens OAuth.
  - PostgreSQL (via SQLAlchemy) : historique des synchronisations.
- Séparation des responsabilités selon la nature des données.

4. Architecture backend

Passage d’une architecture MVC simple à une architecture modulaire en couches distinctes :

- Controllers : gestion des routes HTTP
- Services : logique métier et orchestration
- Models d’intégration : communication avec Spotify et YouTube
- Accès aux bases de données (MongoDB et PostgreSQL via SQLAlchemy)